

BUILD VARIANT B

Variant B - Self-Designed 3D-Printed Chassis

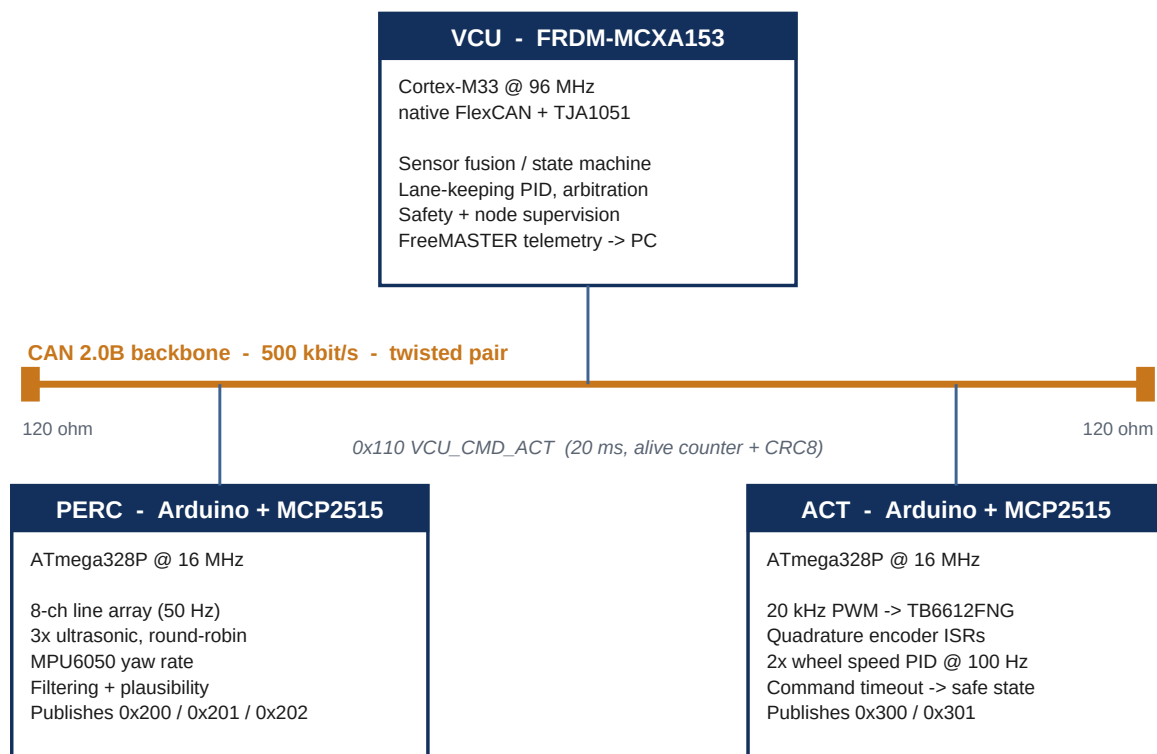
Three-ECU CAN autonomous vehicle on a chassis and wheels designed and printed in-house. Higher schedule risk, but the mechanical design becomes owned intellectual property rather than a purchased part.

1. System overview

A small autonomous vehicle built as **three independent electronic control units** communicating over a single classic CAN bus. The vehicle performs lane keeping (line following), obstacle detection and emergency braking. The target demonstration is not the driving itself but **fault-injection resilience**: a node is physically disconnected mid-lap and the vehicle degrades safely instead of crashing.

Why distribute at all? At this scale one MCXA153 could run everything - it sits at roughly 15 % load. The genuine justifications are timing isolation (ultrasonic ranging blocks for up to 30 ms and must stay away from a 100 Hz control loop), fault containment, I/O locality, and scalability: a fourth node costs one CAN stub, not a redesign. State this honestly in the presentation - it is a stronger answer than pretending CPU is the bottleneck.

2. Node architecture



Estimated bus load: 200 frames/s x ~130 bit = ~26 kbit/s = 5.2 %

Bus constraint. The MCP2515 is **classic CAN 2.0B only - it has no CAN-FD support**. The whole bus therefore runs classic CAN at 500 kbit/s and the MCXA153 FlexCAN is configured in non-FD mode, even though the silicon supports FD. CAN-FD is the version-2 upgrade path once the Arduino nodes are replaced.

3. Electronics and networking - bill of materials

Prices are indicative single-unit EU retail, excluding shipping. This section is **identical in both build variants**.

3.1 Compute and CAN networking

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
1	FRDM-MCXA153 development board	1	VCU node - Cortex-M33, native FlexCAN	17.00	17.00	Have
2	Arduino UNO R3 / Nano	2	PERC and ACT nodes	12.00	24.00	Have
3	TJA1051T/3 or SN65HVD230 transceiver breakout	1	CAN PHY for the MCX native FlexCAN	4.00	4.00	Buy
4	MCP2515 + TJA1050 CAN module	2	SPI CAN controller - AVR has no CAN core	4.00	8.00	Buy
5	120 ohm 0.25 W resistor	2	Bus termination, two physical ends only	0.10	0.20	Buy
6	USB-CAN analyzer (CANable / USBtin) + SavvyCAN	1	Bus sniffing, DBC decode, logging, fault injection	18.00	18.00	Buy

3.2 Sensors

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
7	QTR-8A reflectance array (or 5x TCRT5000)	1	Line / lane position, analog	10.00	10.00	Buy
8	HC-SR04 ultrasonic (or VL53L0X ToF)	3	Obstacle distance left / centre / right	2.00	6.00	Buy
9	MPU6050 IMU	1	Yaw rate and tilt, I2C	3.00	3.00	Buy

3.3 Power and protection

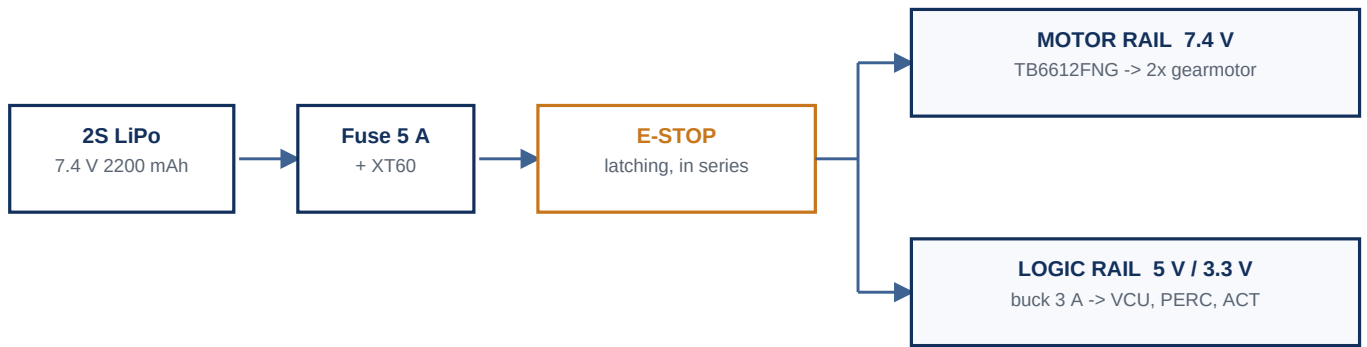
#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
10	LiPo 2S 7.4 V 2200 mAh + balance charger	1	Traction and logic supply	25.00	25.00	Buy
11	Buck converter 5 V / 3 A (MP1584 or LM2596)	1	Logic rail, isolated from motor rail	4.00	4.00	Buy
12	Latching emergency-stop button, 10 A	1	Hardware safe state, firmware-independent	5.00	5.00	Buy
13	Bulk cap 470-1000 uF 25 V + 100 nF assortment	1	Motor rail decoupling	3.00	3.00	Buy
14	XT60 connectors + fuse holder + 5 A fuse	1	Battery interface and overcurrent protection	4.00	4.00	Buy

3.4 Wiring and prototyping

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
15	JST-XH / JST-GH connector kit + crimp tool	1	Keyed ECU harness - CAN_H, CAN_L, VBAT, GND	12.00	12.00	Buy
16	Protoboard 2x, pin headers, jumper wire	1	Node interconnect and transceiver carriers	8.00	8.00	Buy
17	Silicone wire 22 AWG, heat shrink, cable ties	1	Harness build	6.00	6.00	Buy

Electronics subtotal (items to purchase): EUR 116.20 - the two boards already owned (EUR 41.00 of value) are excluded.

3.5 Power architecture



Single-point (star) ground between motor and logic returns. Bulk 470-1000 uF across the motor rail. E-stop removes motor energy independently of all firmware.

4. Mechanical structure - self-designed and 3D printed

The schedule consequence. 3D printing becomes the **critical path**, because print time is calendar time, not work time. One printer yields roughly two design iterations per day. Plan the mechanical work as a track running in parallel with firmware, and print the small interface-defining parts on day one - never the big plates.

4.1 Mechanical bill of materials

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
B1	JGA25-370 gearmotor, 25 mm, ~150 RPM @ 12 V, integrated hall encoder	2	Drive + wheel speed feedback in one part	14.00	28.00	Buy
B2	TB6612FNG dual motor driver breakout	1	Motor bridge - 1.5 V lower dropout than an L298N	4.00	4.00	Buy
B3	Aluminium hex hub coupler, 4 mm D-shaft to 12 mm hex	2	Removes the highest-risk printed feature - see 4.2	3.00	6.00	Buy
B4	Ball caster / ball transfer unit	1	Third contact point - keeps odometry clean	3.00	3.00	Buy
B5	PETG filament (about 400 g consumed)	1	Chassis decks, brackets, wheel rims	12.00	12.00	Buy
B6	TPU 95A filament (about 80 g consumed)	1	Printed tyres	5.00	5.00	Buy
B7	M3 brass heat-set inserts, 100 pcs + soldering-iron tip	1	Every screw joint - highest-value EUR 10 in the project	10.00	10.00	Buy
B8	M3 screw, nut and brass standoff assortment	1	Deck stack-up and board mounting	10.00	10.00	Buy
B9	Nitrile O-ring, 60 mm (tyre fallback)	4	Backup traction solution if TPU printing fails	0.50	2.00	Buy
			Mechanical subtotal		80.00	

4.2 Motor selection - size for RPM, not torque

Run the numbers before shopping. Target mass about 900 g, acceleration 0.5 m/s², wheel radius 30 mm:

$$T_{\text{wheel}} = (m * a * r) / 2 = (0.9 * 0.5 * 0.03) / 2 = 0.007 \text{ N.m} = 0.07 \text{ kg.cm per wheel}$$

A JGA25-370 produces roughly 10 kg.cm stall and even an N20 produces about 1.5 kg.cm. **Torque is a non-issue at this scale - stop optimising for it.** Select on output speed and on whether an encoder is integrated:

$$\text{RPM}_{\text{target}} = v * 60 / (\pi * D) = 0.5 * 60 / (3.1416 * 0.06) = 159 \text{ RPM for } 0.5 \text{ m/s on } 60 \text{ mm wheels}$$

Option	EUR ea	Assessment
JGA25-370, 25 mm, ~150 RPM @ 12 V, integrated hall encoder	14.00	Recommended. M3 face-mount pattern and a 4 mm D-shaft make bracket design trivial and tolerances forgiving
N20 metal gearmotor + magnetic encoder, 200 RPM	11.00	Compact, but M2 mounting screws and a 3 mm shaft - much less forgiving for a first printed design
TT motor (yellow gearbox)	3.00	Avoid. The output shaft is too sloppy to design a precise mount around

Buy motors with encoders already fitted. Retrofitting encoders to a bare motor is a separate multi-day project and will not fit inside this schedule.

4.3 Drivetrain topology

Two driven wheels plus one ball caster - not four-wheel skid steer. Skid steering scrubs the tyres through every turn, which destroys odometry; and odometry is half of what makes this an autonomous vehicle rather than a line follower.

4.4 Wheel design - two problems, both solvable

Problem A: printed plastic has no grip. PLA on laminate flooring is approximately ice. Ranked solutions:

Approach	Grip	Effort	Verdict
Two-part wheel: PETG rim + printed TPU 95A tyre	Very good	Medium	Primary choice
Groove in the rim + 2-3 nitrile O-rings	Good	Low	Fallback if TPU misbehaves
Cheap 1/18-scale RC tyres + printed hub adapter	Excellent	Low	Costs EUR 8 and gives up design ownership
Silicone cast in a printed mould	Best	High	Out of schedule
Bare PLA or PETG tread	Useless	-	Do not

Print the TPU tyre as a separate ring with a 3 mm wall and 0.15 mm interference fit onto the rim, then a drop of cyanoacrylate. Print slowly (20-25 mm/s) with retraction near zero.

Problem B: the hub will slip on the shaft. This is the single most common failure in printed robots - a printed D-bore rounds off within an hour of driving. **Do not print the shaft interface.** Buy EUR 3 aluminium hex hub couplers and design the wheel around the hex pocket. Zero slip, and it decouples the highest-risk dimension from print tolerance. If printing it anyway: PETG only, D-bore plus 0.1 mm, and an M3 grub screw through a brass heat-set insert bearing on the shaft flat.

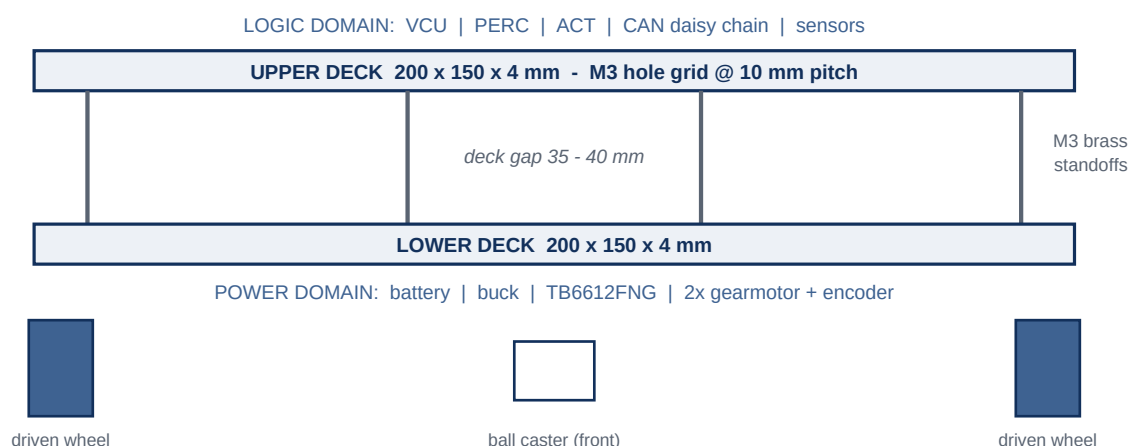
4.5 Chassis design rules

Ordered by how much time each one saves.

- **Heat-set brass inserts (M3, about EUR 10 per 100) at every screw joint.** Printed threads strip on the third assembly, and this vehicle will be assembled thirty times. The highest-value ten euros in the project.
- **A 10 mm grid of M3 holes across the upper deck.** Boards and sensors *will* move. Without the grid that is a four-hour reprint; with it, it is a screwdriver.
- **Design in 2.5D** - extruded 2D profiles printed flat on the bed. No supports, maximum in-plane strength, minimum print time.
- **Parametric from day one.** Define the variables before drawing anything: wheel_dia, track_width, shaft_dia, deck_gap, plate_thk, fit_clearance. Changing wheel diameter must be one number, not a remodel.
- **Print a tolerance test coupon first** - one part, five hole sizes, ten minutes. It calibrates every fit in the project. Skipping it costs a day.
- **The sensor mount is a separate part with slotted holes.** Line-array height (3-8 mm) is a tuning parameter you will adjust constantly. Never make that a reprint.
- **Print cable channels and cable-tie anchors.** Loose jumper wires cause intermittent faults that look exactly like CAN bugs, and half a day gets lost on the wrong layer of the stack.
- **Materials:** PETG for decks, brackets and rims - tougher than PLA and it will not creep under a warm motor. TPU 95A for tyres only. Skip ABS and ASA; warping is not worth it.

4.6 Deck stack-up

Electrical separation of power and logic is mirrored mechanically - one deck each.



4.7 Starting geometry

Adjust these, but do not agonise over them - they are a starting point, and every one is a CAD parameter.

Parameter	Value	Note
Chassis plate	200 x 150 x 4 mm	Fits a 220 x 220 bed with room to spare
Track width (wheel centres)	160 mm	Enters the firmware as TRACK_WIDTH_MM
Wheel outer diameter	60 mm	54 mm printed rim + 3 mm TPU tyre, 25 mm wide
Ball caster position	120 mm ahead of drive axle	Two driven wheels plus one caster - not skid steer
Ground clearance	15-20 mm	Enough for a taped line course and cable routing
Deck gap	35-40 mm	Clears the ECU boards and the CAN harness
Line-sensor lookahead	80 mm ahead of drive axle	Directly sets PID behaviour - make it adjustable
Line-sensor height	5 mm above floor	Slotted mount, tunable 3-8 mm

4.8 Printed parts list and time budget

Part	Qty	Material	Infill	Print time	Note
Tolerance test coupon	1	PETG	20 %	0 h 10	Print first. Five hole sizes - calibrates every fit in the project
Motor bracket	2	PETG	40 %	0 h 45	M3 face mount. Interface frozen on day 3
Wheel rim, 54 mm	2	PETG	30 %	1 h 30	Hex pocket for the aluminium hub, groove for the O-ring fallback
Tyre ring, 3 mm wall	2	TPU 95A	20 %	1 h 00	Print at 20-25 mm/s, retraction near zero, 0.15 mm interference fit
Lower deck, 200 x 150 x 4	1	PETG	30 %	5 h 00	Flat on the bed, no supports. Print last, once dimensions are locked
Upper deck + M3 grid @ 10 mm	1	PETG	30 %	4 h 00	The grid is why you never reprint a deck to move a board
Sensor mast, slotted	1	PETG	30 %	0 h 30	Line-array height adjustable 3-8 mm without a reprint
Battery cradle	1	PETG	25 %	0 h 45	Retains the LiPo under braking
Cable channel clip	6	PETG	25 %	0 h 20	Loose wiring causes faults that look like CAN bugs
Total print time				~16 h	Roughly two design iterations per day on one printer

- **Hour one of day one:** print the tolerance coupon and one motor bracket. These two parts define every downstream interface.
- **Day three: freeze the mechanical interfaces** - bracket, hub, hole grid, deck gap. After that, iterate only on brackets and mounts, never on the plates.
- **Mock the layout in cardboard on day one** before committing to CAD. Twenty minutes there prevents a five-hour reprint.
- **If the school has a laser cutter**, cut the flat decks from 4 mm plywood or acrylic instead - five minutes rather than five hours - and 3D print only the brackets, hubs and wheels. That hybrid is the smart play.
- **CAD tool:** Onshape if working as a team (browser-based, real version control, nothing to install) or Fusion 360 if solo. Parametric modelling is not optional here.

4.9 Where the mechanics meets the firmware

CAD parameters become firmware constants. Wire them together explicitly in a shared header so they cannot silently drift apart:

```
/* veh_geometry.h - derived from CAD parameters, update together */
#define WHEEL_DIA_MM 60.0f
#define ENC_CPR_MOTOR 7 /* hall pulses per motor rev, per channel */
#define GEAR_RATIO 34 /* motor revs per output rev */
#define QUAD_DECODE 4 /* 1x, 2x or 4x edge counting */
#define TICKS_PER_REV (ENC_CPR_MOTOR * GEAR_RATIO * QUAD_DECODE)
#define MM_PER_TICK ((3.14159f * WHEEL_DIA_MM) / TICKS_PER_REV)
#define TRACK_WIDTH_MM 160.0f /* differential mixing constant */
```

Sizing rule for the encoder ISR load. $\text{interrupts/s} = \text{TICKS_PER_REV} \times (\text{RPM} / 60) \times 2 \text{ motors}$. At 150 RPM with the values above that is about 4800/s, roughly 2.5 % of a 16 MHz AVR - comfortable. A high-ratio gearbox (1:100 or more) pushes it past 15000/s and it starts to hurt. If that happens, drop to **1x decoding** (count one edge of channel A, sample channel B for direction): you lose resolution you do not need and reclaim three quarters of the load.

4.10 Ten-day build schedule - two parallel tracks

Printing is idle time you should be coding through. The two tracks converge on day 4.

Day	Milestone
1	Mech: cardboard mock-up, CAD parameters defined, tolerance coupon and motor bracket printed. FW: blink and UART on all three boards.
2	Mech: wheel rim v1, hub fit test. FW: two-node CAN loopback. Nothing else until frames flow.
3	Mech: freeze all mechanical interfaces. Tyres printed, lower deck started. FW: all three nodes on the bus, .dbc decoding.
4	Mech: lower deck assembled, motors mounted, wheels fitted. FW: ACT drives open-loop from a CAN command.
5	Mech: upper deck printed, boards mounted, harness crimped. FW: ACT closed-loop speed PID via FreeMASTER.
6	Mech: sensor mast, height tuning. FW: GATE - the vehicle follows a line.
7	Obstacle detection, emergency braking, vehicle state machine.
8	E2E alive counter and CRC8, timeout supervision, degraded mode.
9	Fault-injection test campaign. Cosmetic v2 parts only if everything above is green.
10	Poster, video, demo rehearsal.

If the school runs only one week, stop at the day-6 gate and spend the remaining time on safety and fault injection. **Cosmetics are day nine or never** - a functional flat plate that demonstrates reliably beats a beautiful chassis with untested firmware.

5. Communication structure - CAN message set

Classic CAN 2.0B, 500 kbit/s, standard 11-bit identifiers. **Lower identifier = higher priority.** Design and freeze this table before writing any application code, and express it as a real **.dbc** file so the analyzer decodes named signals live.

ID	Message	Producer	Period	DLC	Payload
0x080	EMERGENCY_STOP	any node	event	1	reason code - highest priority on the bus
0x100	VCU_STATUS	VCU	100 ms	2	vehicle state, alive counter
0x110	VCU_CMD_ACT	VCU	20 ms	8	speed_sp i16, yaw_rate_sp i16, mode, alive_ctr, crc8
0x200	PERC_DIST	PERC	50 ms	6	dist_left / dist_centre / dist_right, u16 mm
0x201	PERC_LINE	PERC	20 ms	4	line_pos i16 (-1000..+1000), valid_flags
0x202	PERC_STATUS	PERC	100 ms	3	alive_ctr, per-sensor health bits
0x300	ACT_FEEDBACK	ACT	20 ms	8	speed_meas_L, speed_meas_R, odometry ticks
0x301	ACT_STATUS	ACT	100 ms	3	alive_ctr, fault bits, driver temperature flag

Bus load. About 200 frames/s x ~130 bit worst case = ~26 kbit/s = **5.2 % of 500 kbit/s**. Automotive practice keeps a bus below 40-50 %, so there is ample headroom for a fourth node. Put this calculation on the poster - it shows the bus was sized, not guessed.

6. Safety concept

A miniature functional-safety argument. Safe state is defined as: **motors de-energised, brake applied, hazard LEDs on, buzzer active.**

Hazard / fault	Mechanism
Corrupt or stale command	E2E profile on 0x110: alive counter + CRC8; ACT rejects and counts violations
Node silent / crashed	Timeout supervision on every node. ACT: no valid command for 100 ms -> safe state
Perception lost	VCU: no PERC frame for 150 ms -> DEGRADED, speed capped at 30 %
Firmware hang	Watchdog on all three (WDOG on MCX, WDT on AVR)
Any hazard, operator-triggered	Latching hardware E-stop in the motor supply, independent of firmware
Implausible sensor reading	Range and rate-of-change checks; sensor marked invalid in health bits
Bus flooded / rogue frame	0x080 EMERGENCY_STOP holds the lowest ID, so it always wins arbitration

6.1 Fault-injection test plan

This is the demonstration that distinguishes the project from a hobby robot. Rehearse every row.

Injected fault	Expected observable behaviour
Unplug PERC mid-lap	VCU enters DEGRADED within 150 ms, speed capped, vehicle keeps lane
Short CAN_H to CAN_L	All nodes lose comms, ACT times out, motors de-energised within 100 ms
Remove a termination resistor	Increased error frames visible in the analyzer; log and demonstrate
Brown out the 5 V rail	Watchdog reset, node re-enters INIT and re-joins the bus
Inject a rogue frame from the analyzer	E2E check fails, frame discarded, violation counter increments
Corrupt the CRC deliberately	ACT holds last valid command briefly then falls to safe state

7. Software structure

The VCU is built with the MCUXpresso SDK in C, starting from the *flexcan_interrupt_transfer* example for the board. Use **MCUXpresso Config Tools** to mux CAN0_TX / CAN0_RX onto header pins and to compute bit timing - do not calculate the segments by hand. Begin with a 1 ms time-triggered scheduler rather than an RTOS: it is deterministic, trivially explainable, and can be ported to FreeRTOS later if time allows.

Use NXP FreeMASTER. It provides live variable scoping and on-the-fly PID tuning over the MCU-Link VCOM. Tuning a PID by edit-and-reflash will consume the entire schedule; FreeMASTER turns it into a slider.

Node	Module	Responsibility
VCU	can_if.c	FlexCAN init (classic, 500k), RX filters, TX mailboxes
	pdu_router.c	Signal pack/unpack, E2E alive counter + CRC8 verify
	fusion.c	Distance/line filtering, obstacle classification
	vehicle_sm.c	INIT / SELF_TEST / IDLE / ARMED / DRIVE / DEGRADED / SAFE_STOP
	path_ctrl.c	Lane-keeping PID (line_pos to yaw_rate) + speed profile
	safety_mon.c	Per-node timeout table, degraded and safe-state logic
	telemetry.c	LPUART to FreeMASTER over the MCU-Link VCOM
	sched.c	1 ms SysTick time-triggered task table
PERC	us_driver.cpp	Non-blocking ultrasonic FSM, micros() + pin-change ISR
	line_array.cpp	8-channel ADC sweep, calibration, centroid to line_pos
	filters.cpp	Median-of-3 + IIR, range plausibility, health bits
	can_node.cpp	MCP2515 driver wrapper, periodic TX scheduler
ACT	motor.cpp	Timer1 20 kHz PWM, TB6612FNG direction logic, slew limit
	encoder.cpp	Quadrature decode ISRs, tick accumulation, speed estimate
	speed_pid.cpp	Per-wheel PI(D) @ 100 Hz, anti-windup
	kinematics.cpp	(speed, yaw_rate) to per-wheel setpoints, odometry
	safe_state.cpp	Command timeout, fault latch, coast/brake sequence
shared	veh_can_ids.h	Message IDs, signal offsets, scaling - identical on all 3
	veh_geometry.h	Wheel diameter, track width, ticks-per-rev, mm-per-tick
	e2e_crc8.c/.h	CRC8 and alive-counter helpers, host-testable

8. Computational load distribution

Node	Core	Task	Rate	Load
VCU	Cortex-M33 @ 96 MHz	CAN RX/TX, E2E check, signal unpack	event	3 %
		Sensor fusion and filtering	50 Hz	3 %
		Lane-keeping PID + speed profile	50 Hz	2 %
		State machine + node supervision	100 Hz	2 %
		FreeMASTER / UART telemetry	20 Hz	5 %
		VCU total		~15 %
PERC	ATmega328P @ 16 MHz	Ultrasonic round-robin (non-blocking FSM)	20 Hz	5 %
		8-channel ADC line sweep (~0.9 ms)	50 Hz	5 %
		Filtering, plausibility, SPI/CAN TX	50 Hz	10 %
		PERC total		~20 %
ACT	ATmega328P @ 16 MHz	Quadrature encoder ISRs	~4.8 kHz	3 %

Node	Core	Task	Rate	Load
		2x wheel speed PID	100 Hz	2 %
		CAN RX + timeout supervision	50 Hz	6 %
		ACT total		~12 %

Headroom on the VCU is deliberate - it is where stretch goals live (Kalman filter, occupancy grid, Stanley controller).

9. Risk register

Risk	Severity	Mitigation
MCP2515 crystal mismatch	High	Modules ship with 8 MHz or 16 MHz crystals. Wrong setting = zero frames and no error. Read the crystal marking before wiring.
Triple bus termination	High	MCP2515 modules carry an onboard 120 ohm. Three nodes = 40 ohm = dead bus. Remove it on the middle node.
Motor-induced brownout	High	Separate rails, star ground, bulk capacitance, flyback protection. Most common cause of unexplained resets.
Blocking ultrasonic reads	Medium	pulseIn() stalls up to 30 ms. Must be interrupt-driven from day one.
Loose Dupont wiring	Medium	Intermittent faults that look exactly like CAN bugs. Crimp a proper harness.
Scope creep (camera vision)	Medium	Not feasible on these MCUs. If wanted, add an ESP32-CAM as a fourth node publishing lane offset - a stretch goal, never a dependency.

10. Path to a viable product at small scale

The vehicle is not the product - the teaching platform is: a three-ECU automotive networking lab kit for universities, hackerspaces and corporate embedded training. This variant has the stronger commercial story, because the mechanical design is owned rather than resold: the narrative becomes *printed prototype to moulded product* instead of *we repackaged somebody else's kit*.

Step	Action	Why it matters
1	Replace both Arduinos with MCX A14x nodes	One toolchain, one codebase, native CAN-FD. About EUR 1-2 per MCU at volume against EUR 12 per Arduino
2	One generic ECU node PCB in two stuffing variants (PERC / ACT)	Two-layer board at roughly EUR 2 each in quantity 10; replaces breadboard wiring
3	Keyed JST-GH 4-pin daisy chain (CAN_H, CAN_L, VBAT, GND)	Removes the single largest source of intermittent field faults
4	CAN bootloader on every node	Field firmware update over the bus - the biggest single value-add, and a great feature to build
5	End-of-line test fixture: pogo-pin jig that flashes, self-tests and stamps a serial	Required the moment more than about ten units are built
6	Curriculum package: lab manual, .dbc, reference firmware, exercise set	The margin is in the curriculum, not the hardware
7	Printed prototype now, injection-moulded deck above ~1000 units	You own the mechanical design. 3D printing below 50 units, moulding only in the thousands - and the CAD is an asset, not a purchase order

Regulatory gates for EU sale. Sold as a kit, the assembled product falls under the EMC Directive. **Ship without the LiPo** to avoid UN38.3 transport testing - standard practice for drone and robot kits. License the hardware CERN-OHL-S and the firmware Apache-2.0: community adoption is the marketing channel for a platform like this.

Total project cost, Variant B: EUR 196.20 to purchase (electronics EUR 116.20 + mechanics EUR 80.00), on top of the FRDM-MCXA153 and two Arduinos already owned. That is EUR 41.00 more than Variant A, in exchange for owned mechanical design, integrated encoders and better odometry.